kubectl : contextes, hiérarchie des objets et `rollout`

`kubectl` est un simple client : il parle à l'API server d'un cluster Kubernetes. Pour savoir **où** envoyer ses commandes, il lit un fichier de configuration local. Cette fiche distingue deux hiérarchies qu'on confond facilement : celle du **cluster** (ce qui tourne) et celle de **kubectl** (comment on s'y connecte, avec le *contexte*). Elle présente aussi `kubectl rollout status` et `kubectl rollout restart`.

> Le script cité ici, `scripts/test-k8s-deploy.sh`, vit dans le **dépôt Java** (`dev-appli-java`). Le dépôt Angular ne contient que deux manifestes `k8s/` à compléter (voir la fiche sur l'orchestration) : il n'a ni script de test ni `rollout` à lancer. Les commandes `kubectl` sont les mêmes pour les deux dépôts.

Le contexte kubectl

Un **contexte** est un triplet de références stocké dans `~/.kube/config` (le *kubeconfig*) :

Élément	Rôle
cluster	adresse de l'API server et certificat pour le vérifier
user	identifiants (certificat client, jeton)
namespace	namespace utilisé quand on n'écrit pas `-n`

Le fichier contient trois listes (`clusters`, `users`, `contexts`) ; un contexte pointe vers une entrée de chacune. Le **contexte courant** (`current-context`) est celui que toute commande utilise.

```
kubectl config get-contexts          # liste, * = courant
kubectl config current-context
kubectl config use-context minikube  # change le contexte courant
kubectl --context=minikube get pods  # contexte pour une seule commande
```

`use-context` ne touche **pas** au cluster : il réécrit seulement `current-context` dans `~/.kube/config`, donc le changement vaut pour tous les terminaux.

Le contexte `minikube`

`minikube start` crée automatiquement un cluster, un user et un contexte, tous nommés `minikube` :

```
contexte "minikube"
├── cluster   → server: https://127.0.0.1:41758, CA : ~/.minikube/ca.crt
├── user      → client.crt / client.key dans ~/.minikube/profiles/minikube/
└── namespace → default
```

L'adresse `127.0.0.1:<port>` est un port redirigé vers le conteneur Docker de Minikube (driver `docker`) ; ce port peut changer à chaque démarrage et Minikube met alors le fichier à jour. Avec un seul profil Minikube, il n'y a **qu'un cluster** et un contexte. Un second profil (`minikube start -p autre`) ou un cluster cloud ajouterait des entrées, et `use-context` servirait à choisir.

Qui contient qui ?

Côté cluster (ce qui tourne) :

```
Cluster
└── Namespace           cloisonnement logique (default, workshop-test-1234…)
    └── Deployment      « je veux N pods de cette image »
        └── ReplicaSet
            └── Pod     un ou plusieurs conteneurs
                └── Conteneur   l'application ou la base
```

Une « application » n'est pas un objet Kubernetes : c'est un ensemble de ressources (Deployment, Service, PVC…) regroupées par un namespace et par des labels. Certains objets sont au niveau du cluster et non d'un namespace : les nœuds, les namespaces eux-mêmes et les PersistentVolumes.

Côté kubectl (comment on s'y connecte) :

```
~/.kube/config
├── clusters: [minikube]
├── users:    [minikube]
└── contexts: [minikube]  → référence 1 cluster + 1 user + 1 namespace par défaut
```

Le contexte **ne contient aucune ressource Kubernetes** : il désigne un cluster, une identité et un namespace, et il existe dans un fichier de votre machine, pas dans le cluster.

Action	Effet
`kubectl config delete-context minikube`	supprime le raccourci ; **rien** n'est détruit dans le cluster
`kubectl delete namespace X`	supprime tout le contenu du namespace (pods, Deployments, Services, PVC) ; le contexte n'est pas modifié

`kubectl rollout`

Un *rollout* est le remplacement progressif des pods d'un Deployment (stratégie `RollingUpdate` par défaut).

Commande	Nature	Effet
`kubectl rollout status deploy/X`	observe	suit l'avancement et **bloque** jusqu'à la fin ; code de sortie `0` si succès, non nul si échec ou dépassement de `--timeout`
`kubectl rollout restart deploy/X`	agit	déclenche un nouveau rollout **sans changer le manifeste**
`kubectl rollout undo deploy/X`	agit	retour à la révision précédente
`kubectl rollout history deploy/X`	observe	liste les révisions

`restart` ajoute l'annotation `kubectl.kubernetes.io/restartedAt` au modèle de pod : la spec change, donc les pods sont remplacés. C'est utile quand une ConfigMap ou un Secret a changé (les variables d'environnement ne se rechargent pas seules), quand une image à tag mutable (`latest`) a été repoussée, ou pour relancer des pods bloqués. Les deux commandes se combinent :

```
kubectl rollout restart deployment/backend
kubectl rollout status deployment/backend   # attendre la fin et confirmer
```

Dans DataShare

Le script `scripts/test-k8s-deploy.sh` du dépôt Java met ces notions en pratique :

- **Garde-fou sur le contexte** : il vérifie que `kubectl config current-context` vaut `minikube` et sinon s'arrête (code 2) en affichant `kubectl config use-context minikube`. Il ne change pas de contexte lui-même, car il **crée et supprime** des ressources : sur un contexte pointant vers un autre cluster, il créerait puis supprimerait un namespace de test au mauvais endroit, et changer le contexte modifierait votre configuration globale à votre insu.
- **Namespace jetable** : il crée `workshop-test-$RANDOM` et passe `-n "$NS"` à chaque commande, au lieu de s'appuyer sur le namespace `default` du contexte. Le test n'interfère donc pas avec ce qui est déployé ailleurs, et le `cleanup` (`kubectl delete namespace`) supprime aussi le PVC (PersistentVolumeClaim) de la base.
- `rollout status` **comme assertion** : `check "la base est prête (rollout)" kubectl rollout status -n "$NS" "$DB" --timeout=120s` s'appuie sur le code de sortie. Il est relancé après la suppression d'un pod pour attendre que Kubernetes l'ait recréé.
- Dans le dépôt Angular, deployment-app.yaml et service-app.yaml sont encore des squelettes : un `rollout status` n'aurait de sens qu'une fois l'image renseignée.

À retenir

1. Un **contexte** = cluster + user + namespace par défaut, enregistrés dans `~/.kube/config`. Il désigne une cible, il ne contient rien du cluster.
2. `use-context` modifie un fichier local, pas le cluster ; supprimer un contexte ne détruit rien, supprimer un namespace détruit tout son contenu.
3. Hiérarchie du cluster : cluster › namespace › Deployment › Pod › conteneur ; l'« application » est un ensemble de ressources, pas un objet.
4. `rollout status` **observe** (et sert d'assertion grâce à son code de sortie), `rollout restart` **agit** (nouveau rollout sans changer le manifeste).
5. Un script qui crée et supprime des ressources doit vérifier le contexte courant et travailler dans un namespace jetable.